# Exploring Bad Code:

## Red Team

Alea Lyn Maxwell

Department of Cybersecurity, Utah Valley University

CYBR 4550: Threat Hunting and IR

Prof. Andrew Haro

October 31, 2026

# Table of Contents:

**Table of Contents::** 1

**Executive Summary:** 2

**Scope and Methodology:** 3

**Findings or Remediations:** 4

**Git Hub Link:** 18

**Appendices:** 18

**AI Usage Statment** 20

**References** 21

# Executive Summary:

There are several vulnerabilities in this website. The risk of this ranges from hacks that could cause the entire website to collapse, having all of our users' data stolen, or having users no longer trust our website at first glance. There are currently eight major vulnerabilities found on this website right now. If we were to roll out this website professionally it could cause several issues. The website needs at least 2-3 more months to fix these vulnerabilities before it should be rolled out because many of these are critical or high.

If we don’t fix these vulnerabilities the company could end up having to pay thousands of dollars down the line to fix these vulnerabilities. In addition some of them could cause us to lose the website entirely. What I need from the board is 3 guaranteed months to fix the highest risk vulnerabilities that are the most pressing. I will also need access to the website's code because there is a chance I might have to reprogram parts of the website depending on certain vulnerabilities that may be lurking in the code.

# Scope and Methodology:

My scope is the birthday memory repo on git hub. Any other fields in the repo besides the one that are assigned to this folder are out of my scope so I will not be working with them. My methodology was OWASP Web Security Testing Guide (WSTG). I felt comfortable using this because I have used them before. For this I used the tool chain of curl(it can help with fuzzing), OWASP Zap (it can do a lot and covers interception and fuzzing as well as detecting OWASP top 10), nmap (helps identify ports and services that are exposed to threat actors), trivy (collects mass scans of docker images, filesystems, and secrets), semgrep (pre existing owasp-aligned rules), psql (this is to examine the database itself and examining the website myself. These tools help me see what vulnerabilities have and help me hit the main vulnerabilities of the website, database, and the APIs. My limitations are I only have the files given. I also only have so much time to spend on this, you can look forever for things that you may never find which causes a huge limitation for me that I don't have infinite time.

# Findings or Remediations:

**Proxy Address Vulnerable to IP Spoofing - Critical**

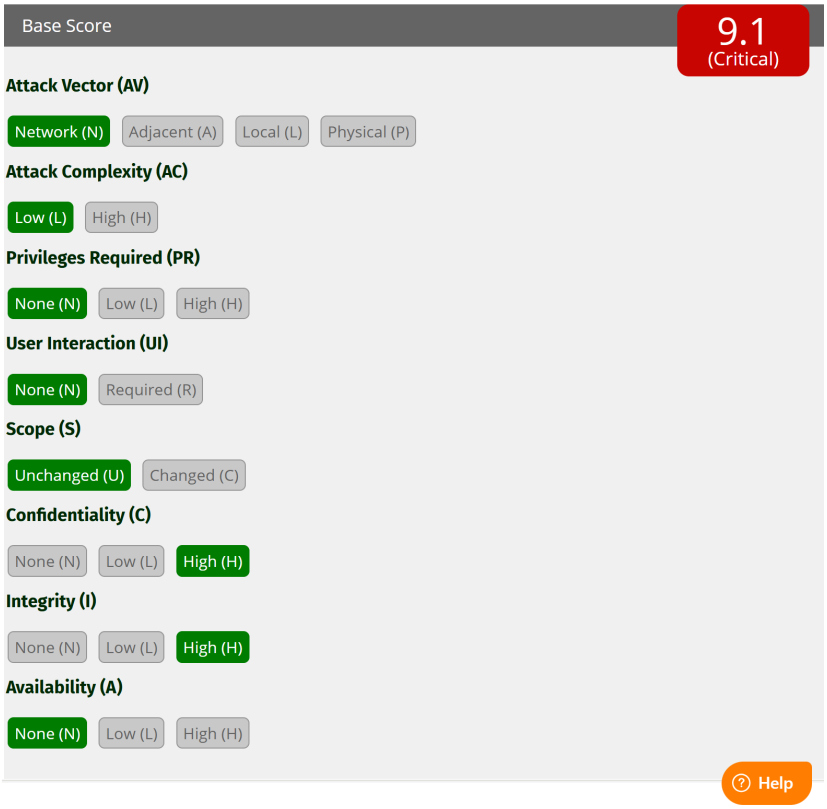
This vulnerability is known as CVE-2026-90711. This is where every unauthenticated client is trusted as the proxy so it returns what anyone sends in as 'X-Forwarded-For" and there is no IP-based control.

To reproduce this vulnerability you need to run Trevy. The screenshot below shows the vulnerability.

| | | | | | |
|---|---|---|---|---|---|
| proxy-addr | CVE-2026-90711 | CRITICAL | 2.0.7 | 2.0.8 | proxy-addr: proxy-addr vulnerable to IP spoofing via IPv4-mapped IPv6 trust subnet<br>https://avd.aquasec.com/nvd/cve-2026-90711 |

Screenshot of the Proxy Address Vulnerable to IP Spoofing vulnerability.

The CVSS score of this is below.




The CVSS score of the Proxy Address Vulnerable to IP Spoofing vulnerability taken from first.org. The reason why this vulnerability has its score is because it attacks on the network, the complexity is low and it doesn't take much work to exploit it. There are also no privileges needed or interaction to get it. It doesn’t change the scope because it only affects the items managed by the company and the confidentiality is high because we know that this vulnerability is there. It does affect our businesses' integrity because it's a total loss of privacy. Finally there is no impact availability.

The business impact of this vulnerability is a potential loss of data. This loss of data could also trickle down into visitors to our website thus lessening network traffic and that could cause a financial loss.

My recommendation is to fix this vulnerability by upgrading the proxy-addr to proxy-addr 2.0.8 or later. This is the recommended fix professionally. It is on the coding layer.

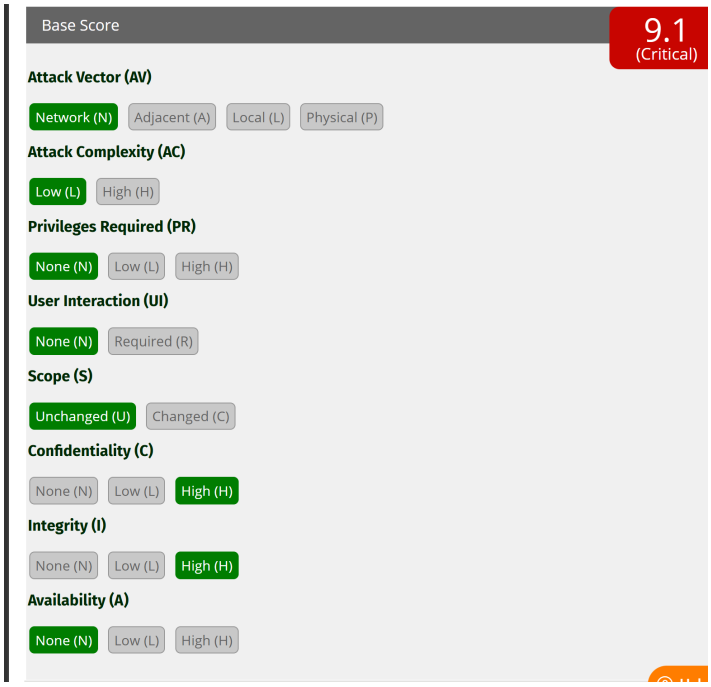
## PII on Main website link - Critical

This vulnerability is as simple as all of our clients email addresses and phone numbers are visible to anyone on the main website.




The website showing all of the Personal identifying information

The screenshot above shows a high risk vulnerability. This is a confirmed vulnerability. This is on the information gathering level of the OWASP Web Security Testing guide. This is created by just going on the website itself.




The CVSS score above for the PII on Main website vulnerability.

I marked it as high impact because its all the data, low complexity because its right there, no privileges because anyone can see it, no user interaction because its in plain view, high integrity impact because all the data is exposed, no availability impact because it doesn't stop people from using the website, and high confidentiality impact because it's right on the main website without any hiding it. You don't have to do anything to access it.

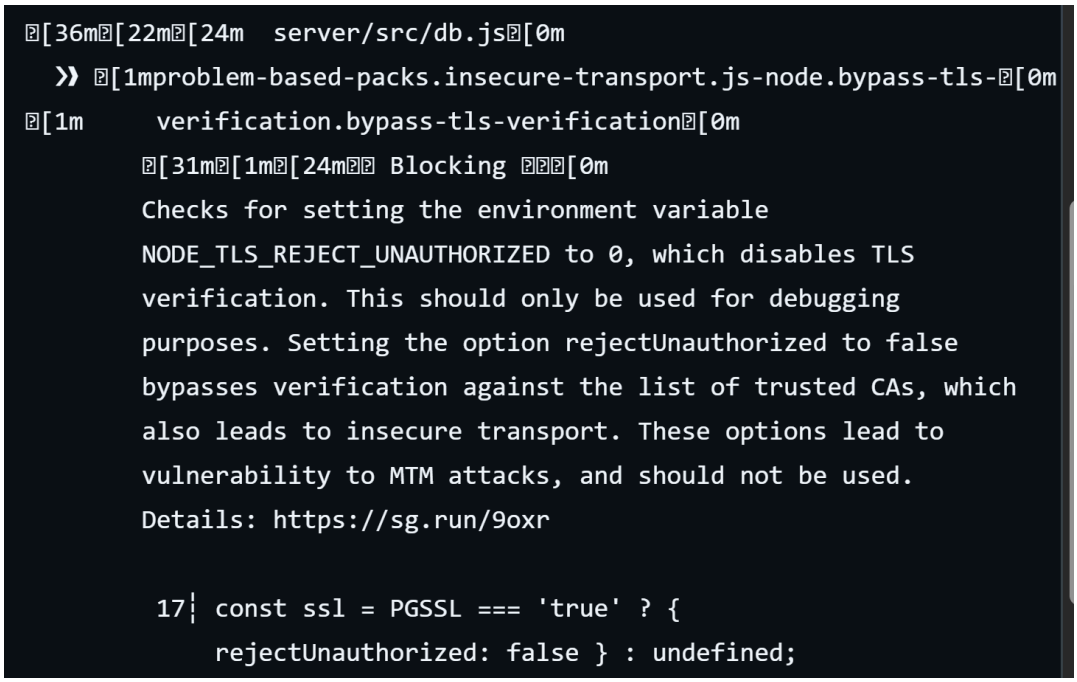
The business impact is that it could cost the website's reputation. No one wants to use a website that puts so much personal data online for everyone. This could cause the company to not even function.

My recommendation is to put the personal information either behind an admin account so only those with authorization can access it or eliminate that aspect as a whole because it's not really necessary for the functionality of the website. This is because there is no other way to really alter what people see and if we want to keep the data so we can easily access it we can add an account to log in. Letting the website sit as it is is not an option. This is done in the code.

**Insecure Transport - High**

This vulnerability means that the Node.js does not reject invalid, untrusted, or self-signed certificates. This means that we are now vulnerable to Man in the Middle attacks that could cause people to lose data to interception from their parties.

To reproduce this vulnerability you need to run Semgrep. The screenshot below shows the vulnerability.




Screenshot of theInsecure Transport vulnerability.

The CVSS score of this is below.




The CVSS score of the Insecure Transport vulnerability created on first.com.

It is a network error, it does require knowledge of doing a man in the middle for complexity, it requires no privileges or user interaction, it does destroy the confidentiality and integrity of the website but it won't affect if the website is available.

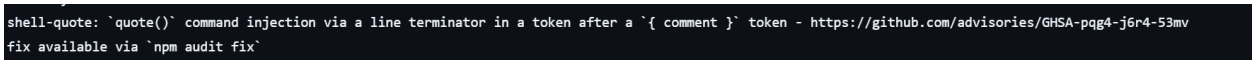
The business impact of this vulnerability is that people could lose their data. If this does lead to a hack it could cost the business thousands of dollars to fix and we would have to report the hack which could cause serious damage to the company's image.

My recommended fix is to turn NODE_TLS_RECJECT_UNOTHARIZED to true. This resolves the issue without changing the main functionality of the website. This is done in the code.

**Proxy Address Vulnerable to IP Spoofing - High**

This vulnerability is where a quote gets rid of a comment and then comments out the rest of the line. This could lead a threat actor to create a controlled string with the line terminator to follow a '{comment}" token in the same call.

To reproduce this vulnerability you need to run npm. The screenshot below shows the vulnerability.




Screenshot of the “Quote()” common injection vulnerability.

The CVSS score of this is below.




The CVSS score of the“Quote()” common injection vulnerability taken from first.com.

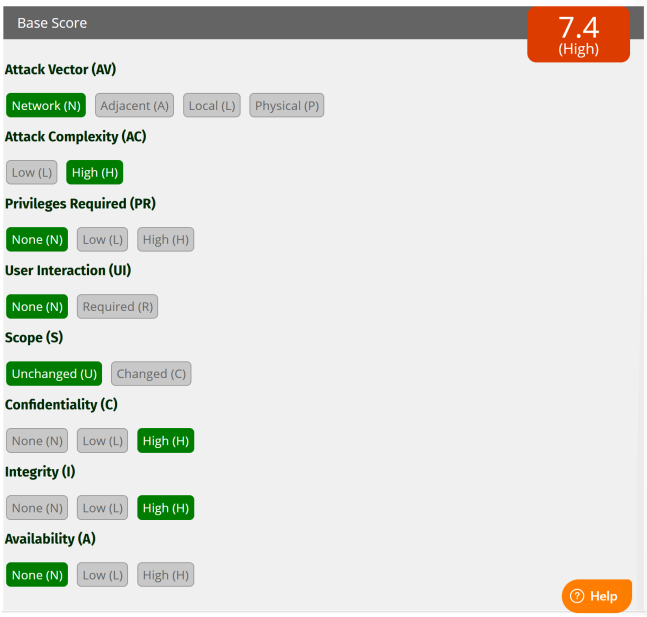
This is a network issue. It does require knowledge of how to do the attack, but doesn't require any privilege or interaction, our scope stays the same, but it completely destroys the confidentiality and integrity of our website and could potentially cause our website to become unavailable as a whole.
The business impact of this vulnerability is that a threat actor could break into the system and mess with the code. This is an expensive fix depending on how much damage they do overall to the system.
My recommendations are to check that all of the open source dependencies have no vulnerabilities and if they do find alternative tools. It also is to reject line terminators in the text but not after. This would be a change in the code. The only other alternative would be to take the risk which is a high risk vulnerability so that would not be recommended.

**Integrity Tag Missing - High**

This vulnerability is that the website is missing an integrity attribute that allows you to verify hosted files. Without this it could cause a threat actor to manipulate the website and could lead to XSS or other attacks.
To reproduce this vulnerability you need to run Semgrep. The screenshot below shows the vulnerability.

```
[36m[22m[24m  client/index.html[0m
  ❯❯ [1mhtml.security.audit.missing-integrity.missing-integrity[0m
        [31m[1m[24m Blocking [0m
        This tag is missing an 'integrity' subresource integrity
        attribute. The 'integrity' attribute allows for the browser
        to verify that externally hosted files (for example from a
        CDN) are delivered without unexpected manipulation. Without
        this attribute, if an attacker can modify the externally
        hosted resource, this could lead to XSS and other types of
        attacks. To prevent this, include the base64-encoded
        cryptographic hash of the resource (file) you're telling the
        browser to fetch in the 'integrity' attribute for all
        externally hosted files.
        Details: https://sg.run/krXA
```

Screenshot of the Integrity Tag Missing via unhandled exception during array serialization vulnerability.
The CVSS score of this is below.




The CVSS score of the Integrity Tag Missing exception during array serialization vulnerability.

The CVSS score comes from the fact that it is a network issue, it does take knowledge of XSS attacks, it doesn' t require privileges, interaction and doesn't change the scope. It will compromise the confidentiality and integrity of the website because data could easily get stolen, and it won't completely take down the website.

The business impact of this vulnerability is that this could cause the website to get hijacked thus losing us the users data thus significantly damaging our reputation.

My recommendation for this vulnerability is to include a cryptographic hash on the website and fetch the integrity attribute for all externally hosted files. The other methods to fix this take longer and this is the main recommendation from professionals. It would be a code change.

**Body Parser Denial of Service - Medium**

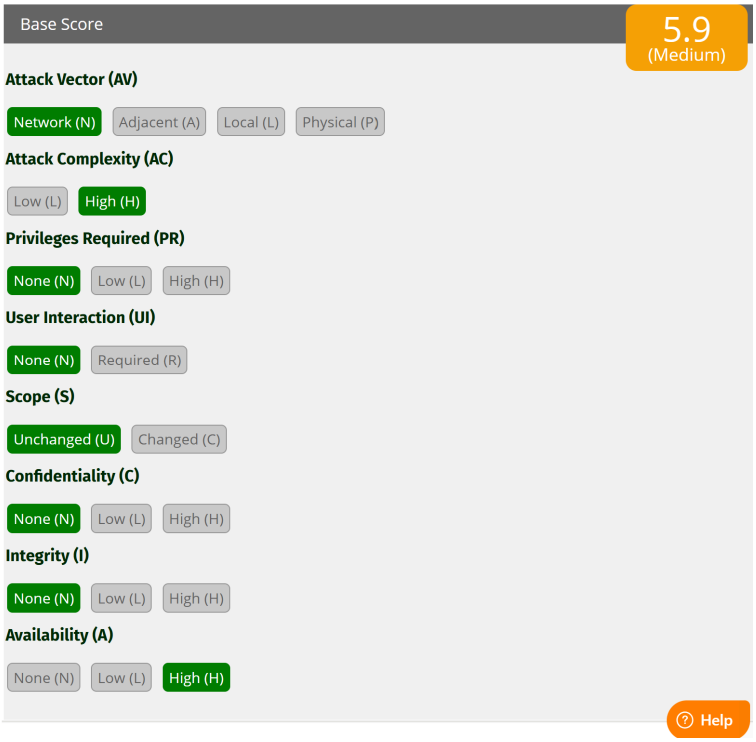
This vulnerability is called qs.stringify when it has an arrayFormat:'comma' that has arrays that don't have a toString method to throw an unhandled error. This can cause failures or crashes on a website. This can also allow remote attacks to supply crafted input and make it look like nothing happened.

To reproduce this vulnerability you need to run Trevy. The screenshot below shows the vulnerability.

| Library | Vulnerability | Severity | Status | Installed Version | Fixed Version | Title |
|---|---|---|---|---|---|---|
| body-parser | CVE-2026-111181 | HIGH | fixed | 1.20.8 | 2.4.0, 1.20.9 | body-parser: body-parser: Denial of Service via unhandled decompression stream error https://avd.aquasec.com/nvd/cve-2026-111181 |

Screenshot of the Body Parser Denial of Service vulnerability.

The CVSS score of this is below.




The CVSS score of the Body Parser Denial of Service vulnerability taken from first.org.

The cvss score comes from it being a network vulnerability, it takes lots of work to craft malware to steal data, it doesn't require privileges, it is just there and doesn't require interaction, it does not change our scope, it won't steal data, and could cause our entire website to no longer be available.

The business impact of this vulnerability is that the website could crash and frustrate many customers causing them to not return to our website. It can also allow a hacker to add more code into the website thus causing more clients data to be stolen which is expensive to fix.

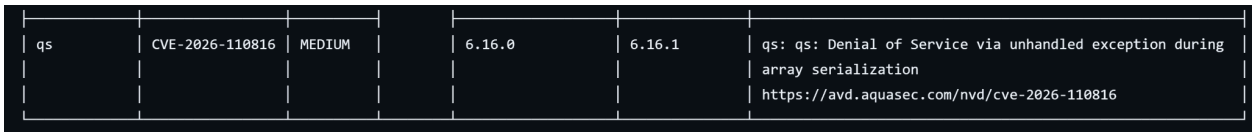
My recommendation to fix this vulnerability is to change the array format from arrayFormat:'comma' to the default indices format. This is the one recommended for these particular vulnerabilities by the industry

and is a doing change. The other alternative would have us change too much about the website's function and would take longer.

**Denial of Service via unhandled exception during array serialization - Medium**

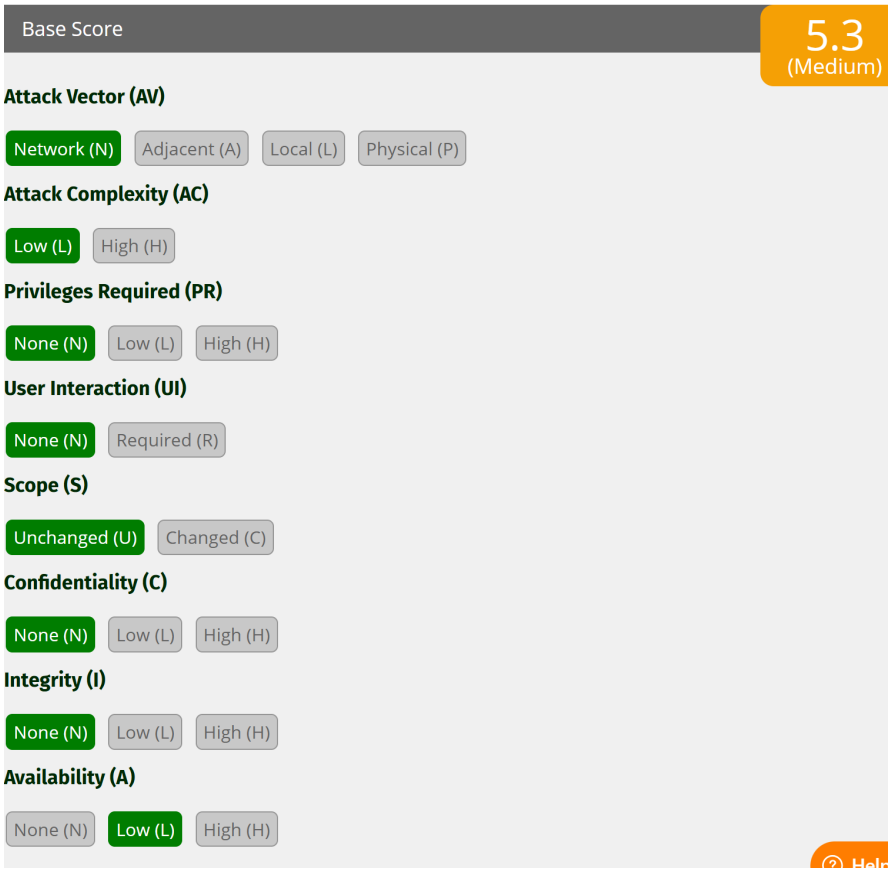
This vulnerability is that the client crashes when Node.js processes a remote WebSocket. This means that a compromised websocket or a malicious one could crash our entire server with one logon.

To reproduce this vulnerability you need to run Trevy. The screenshot below shows the vulnerability.




Screenshot of the Denial of Service via unhandled exception during array serialization vulnerability.

The CVSS score of this is below.



The CVSS score of the Denial of Service via unhandled exception during array serialization vulnerability using first.com.

The CVSS score comes from it being on a network, only taking one corrupted server not code or anything, you just need to log on so it needs no privilege, it doesn't change our scope, it doesn't expose the data or its integrity, and it won't permanently take down the website.

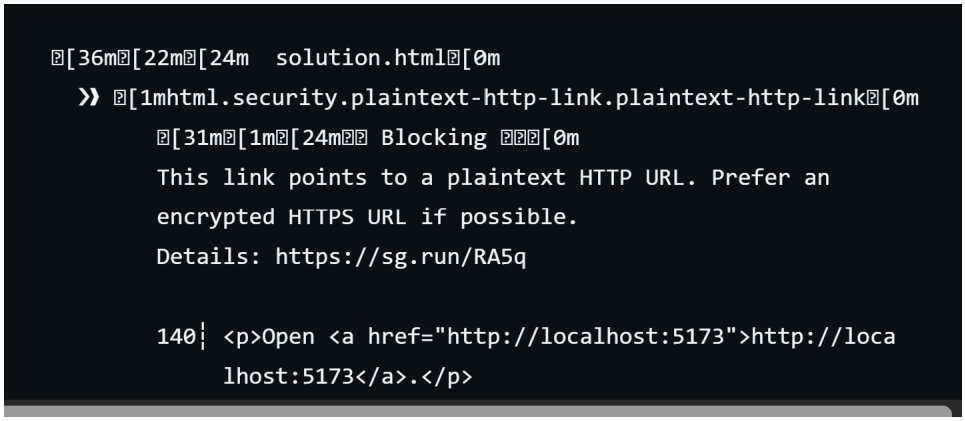
The business impact of this vulnerability is that the website could crash completely thus causing clients to be frustrated and if they get frustrated enough they might not come back.

My recommendation to fix this is to upgrade the version Udicia is running on to being 6.28.1, 7.29.1, 8.10.2 or higher. This is a simple update change and the fastest and easiest way to fix it.

**Plaintext - Low**

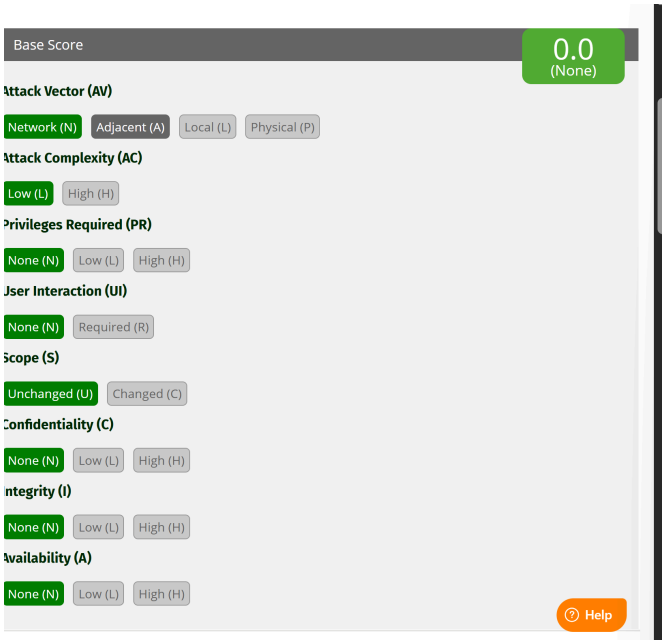
This link is that the website there is not on an encrypted url.

To reproduce this vulnerability you need to run Semgrep. The screenshot below shows the vulnerability.




Screenshot of the Plaintext vulnerability.

The CVSS score of this is below.




The CVSS score of the Plaintext vulnerability.

The CVSS score comes from it being on the website. It doesn't involve any attack complexity, there are no privileges or user interaction, it doesn't change the scope. It also doesn't change the confidentiality or integrity because the data is still safe. It also doesn't cause the website to be unavailable.

The business impact of this vulnerability is that some people might not go onto the website because it's not an https website.

My recommendation is to change the url to be an https by getting an ssl certification or fixing the server based on the issue. This is the best way, the other option is to leave it but that could cause us to lose clients.

# Regulatory and compliance considerations:

The website needs to fall under GDPR which is that you only should collect data that is necessary and for specific purposes. The phone number and email have no purpose of being there. We also need to handle the data for GDPR that ensures appropriate security which means

unauthorized access which having it public grants unauthorized access. Another law to keep in mind is state breach-notification law which states that you need to notify individuals when their personal data is stolen. With the insecurities in the database this could cause issues in the future. Finally the last law to keep in mind is CCPA/CPRA which is the California Consumer Privacy Act which gives individuals the right to delete personal information collected about them. Currently as the website stands this is not possible.

# Recommendations and roadmap:

In the first week you need to update all of your outdated software versions and NODE_TLS_RECJECT_UNOTHARIZED to true. These seem to have the highest rated vulnerabilities with the easiest fixes. These updates should only take a day or two but I gave a buffer just in case and the node is a change that should only take a minute. After that you need to spend the next two weeks to alter the code so the email and phone numbers don't show on the website and are behind a logon account that is secure. This is going to cost more but adds that security and will encourage people to actually use the website. Then in the next month you need to make sure all of your dependencies are secure and change the array structure. These will both take a while and the vulnerabilities are less on the list however you need to make sure that the base of the website is secure because otherwise it could cause a trickle down effect if something happens to the dependencies get hacked that means you might have gotten hacked. Finally work on getting a https certificate next quarter because it only affects how people view the website mostly. This will take the longest but in the long term it will prove that your company is legitimate.

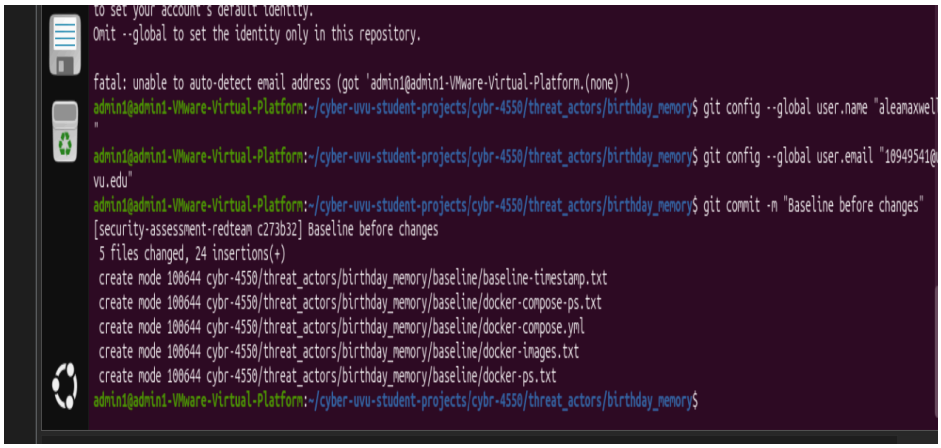
# Conclusion:

I believe that this website should not be shipped out. There are too many high and critical vulnerabilities that leave so much data exposed. Releasing the website now without implementing the fixes could cause a major loss of reputation and could end up costing more money in the long run as well as building back the reputation that this should stay in production until all of the vulnerabilities with the exception of the second quarter vulnerabilities fix are repaired.

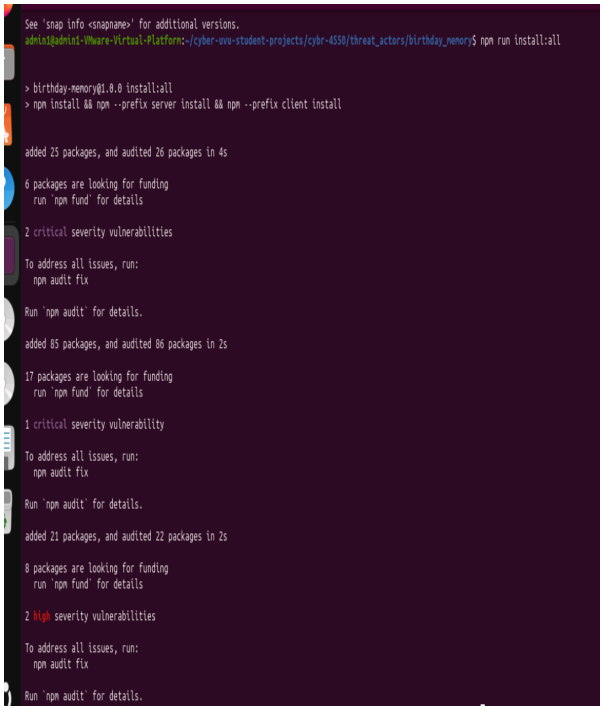
# Git Hub Link:

[cyber-uvu-student-projects/cybr-4550/threat_actors/birthday_memory/security at security-assesment-redteam · aleamaxwell/cyber-uvu-student-projects]

# Appendices (Additional Screenshots):




Baseline before changes created




Dependencies being installed on the vm from the git file.




Health check for the local host

# AI Usage Statment:

As pertaining to the Exploring Bad Code for UVU's Cyber 4550 course the Generative AI Copilot was used with the explicit purpose of :

- Guiding the best tools to use to scan the bad code
- Helping generate and commit the scans onto the badcode and security branch
- Troubleshooting issues with the scans not giving back proper code

All of the security assessments were done by Alea Maxwell.

# References

(n.d.). OWASP Web Security Testing Guide | OWASP Web Security Testing Guide. Retrieved October 30, 2026, from https://wstg.owasp.org/

Barad, A. (2026, 10 27). *CVE-2026-110816: Denial of Service via Uncaught Exception in Node.js qs Library*. https://cvereports.com/reports/CVE-2026-110816

*Common Vulnerability Scoring System Version 3.1 Calculator*. (n.d.). Forum of Incident Response and Security Teams (FIRST). Retrieved October 30, 2026, from https://www.first.org/cvss/calculator/3.1#CVSS:3.1/AV:N/AC:H/PR:N/UI:N/S:U/C:N/I:N/A:H

CVE feed. (2025, 9 15). *CVE-2026-90711 proxy-addr vulnerable to IP spoofing via IPv4-mapped IPv6 trust subnet*. CVE Feed. https://cvefeed.io/vuln/detail/CVE-2026-90711

Gitlab. (2026, 9 28). *CVE-2026-85024: undici vulnerable to Denial of Service via unhandled error in WebSocket permessage-deflate decompression*. Gitlab. https://advisories.gitlab.com/npm/undici/CVE-2026-85024/

GitLab. (2026, 10 5). *CVE-2026-90711: proxy-addr vulnerable to IP spoofing via IPv4-mapped IPv6 trust subnet*. GitLAb. https://advisories.gitlab.com/npm/proxy-addr/CVE-2026-90711/

GitLab. (2026, 10 6). *CVE-2026-102422: shell-quote: `quote()` command injection via a line terminator in a token after a `{ comment }` token*. GitLab. https://advisories.gitlab.com/npm/shell-quote/CVE-2026-102422/

W3Tutorials.net. (2026, 9 30). *Understanding `node_tls_reject_unauthorized` in Node.js*. W3Tutorials.net. https://www.w3tutorials.net/blog/node_tls_reject_unauthorized-nodejs/